

docs/multi-tenant-plan/plan-for-averpo.md

Averpo'ni multi-tenant'ga ўтказиш плани (plan-for-averpo)

Сана.: Ҳолат: ЛОЙИҲА (фойдаланувчи тасдиғи кутилади). Манбалар: plan-a.md (session resolution), plan-b.md (column + RLS), plan-c.md (фазали QBOA) + лойиҳанинг реал инвентаризацияси (§3). Учала план ҳам кодни кўрмасдан ёзилган - бу ҳужжат уларнинг тўғри ғояларини олади, хатоларини Averpo фактларига тузатади (§1).

0. Хулоса (стратегия)

Битта PostgreSQL база, битта схема, `tenant_id` устуни (column-based discriminator) - QBO услуги. Уч қатлам:

1. **Hibernate native `@TenantId`** (ORM 7.x, Spring Boot 4 BOM) - entity сўровлари автоматик филтрланади, INSERT'да автотўлдирилади. plan-b'даги қўлбола `@FilterDef/@Filter` ИШЛАТИЛМАЙДИ (эскирган йўл).
2. **PostgreSQL RLS (FORCE билан)** - defense-in-depth. Бизда ҳисоботлар JdbcClient хом SQL (§3.4, энг катта риск нуқтаси) - RLS уларни ҳам мажбурий қамрайди.
3. **Session-based tenant resolution** (plan-a) - URL'лар ЎЗГАРМАЙДИ, subdomain/JWT КИРМАЙДИ (бизда form login + HttpSession, SPA йўқ).

tenant = компания. Фойдаланувчи (глобал login) бир нечта компанияга аъзо бўлиши мумкин, ҳар бирида ўз роли (QBO: бир Intuit akkaунт - кўп компания). Битта HttpSession = битта фаол компания.

1. Учала планга баҳо

План	Олинади	Рад / тузатиш
plan-a	Session resolution модели тўлиқ: <code>CURRENT_TENANT_ID</code> + <code>TENANT_CONTEXT_VERSION</code> , membership текшируви ҳар сўровда, <code>/tenant/switch</code> , stale-form версия текшируви, endpoint таснифи, HTMX ҳам филтрдан ўтади	<code>/register</code> , <code>/invitation/**</code> ҳозирча йўқ (§9 O1); navbar эмас - бизда sidebar; JTE контекст узатиш нуқтаси аллақачон бор (<code>GlobalModelAttributes</code>)
plan-b	Column-based + RLS ғояси, tenant_id БИРИНЧИ устунли индекслар, connection'да <code>set_config</code>	<code>@FilterDef/@Filter</code> - эскирган (native <code>@TenantId</code> бор); «Flyway (yoki ...)» - бизда Liquibase; subdomain/JWT/header resolution - бизга мос эмас
plan-c	Фазали ёндашув, RLS FORCE нозиклиги, backfill тартиби (nullable → backfill → NOT NULL), unique'ларни тўлиқ қайта қуриш талаби, <code>set_config(?, ?, false)</code> PreparedStatement, scheduler tenant iteration, тест матрицаси, MDC'да tenantId	Flyway → Liquibase; JWT/subdomain → session; «НСБУ» → бизда IFRS; «UZS base» → home currency per-tenant (prod'да USD!); read-replica/Hazelcast/S3 бизда ЙЎҚ (attachment локал диск, кэш умуман йўқ); Testcontainers → локал <code>averpo_test</code> ; «sequence'ни counter жадвалга алмаштир» → бизда АЛЛАҚАЧОН counter жадвал (<code>document_sequence</code> , қатор қулфи билан) - фақат unique'ни tenant'га кенгайтириш қолади

2. Мақсадли модель

- **Tenant** (янги жадвал, глобал реестр): name, status (ACTIVE/SUSPENDED/CLOSED), createdAt, subdomain ЙЎҚ, plan/billing ЙЎҚ (\$8). CompanySettings билан 1:1 - Tenant техник реестр, CompanySettings бизнес-профиль (home currency, timezone, valuation, closing date, реквизитлар) бўлиб қолади, фақат singleton'дан per-tenant'га ўтади.
- **TenantMembership** (янги): user_id + tenant_id + role (мавжуд 8-роль enum) + status. `app_user.role` устуни шу ерга КЎЧАДИ - роль энди компанияга нисбатан (А компанияда SUPER_ADMIN, В'да VIEWER_AUDITOR бўлиши мумкин).
- **Session**: фақат `CURRENT_TENANT_ID` + `TENANT_CONTEXT_VERSION` (plan-а қоидаси - entity сақланмайди).
- **Login оқими** (plan-а): 0 аъзолик → компания яратиш саҳифаси; 1 аъзолик → автотанлов; кўп → танлов саҳифаси.
- **Authorities per-tenant**: ҳозир `JpaUserDetailsService` login'да `RolePermissions.authorities(user.getRole)` беради. Энди authorities фаол компаниянинг membership ролидан қурилади: login / компания танлаш / switch пайтида Authentication қайта қурилиб session'га ёзилади. Битта session = битта фаол tenant бўлгани учун бу тўғри ва арзон - `Perms.current`, `canEdit`, sidebar фильтри, `UrlPermissionMap` ЎЗГАРМАЙДИ.

3. Реал инвентаризация (ҳолати)

Икки чуқур аудит натижаси (домен + инфра):

3.1 Кўлам

- **68 entity / 68 жадвал**, ҳаммаси `BaseEntity` дан (истисно: `PluginState`). Liquibase: охирги changeset **068**, номлаш `NNN-nom.sql`, кетма-кет.
- Хужжат рақамлаш: `DocumentSequence` counter жадвал, `DocumentSequenceService.next` MANDATORY транзакция + қатор қулфи (SELECT FOR UPDATE) - rollback'да рақам қайтади. 14 service чақиради.
- `CompanySettings` singleton: `singleton_guard=true` устунда UNIQUE.
- Кэш ЙЎҚ (@Cacheable/2LC/Caffeine - 0 та). OSIV ўчиқ. Вақтлар UTC.

3.2 Глобал (tenant_id ОЛМАЙДИган) жадваллар

`tenant` (янги), `tenant_membership` (янги), `app_user`, `plugin_state`, `telegram_settings` (\$9 O3), `audit_event` (tenant_id NULLABLE олади - \$4 D10). Қолган **64 жадвал tenant-scoped**.

`currency` ва `exchange_rate` ҳам TENANT-SCOPED (\$4 D3).

3.3 Қайта қуриладиган UNIQUE'лар (тўлиқ рўйхат Т1 картасида)

Ҳозир ҲАММАСИ глобал. `(tenant_id, ...)` композитга ўтадиганлар: - Каталоглар: `currency.code`, `account.name / code` (partial), `item.name / sku` (partial), `item_category.name`, `unit.name`, `unit_group.name`, unit base-partial, `warehouse.name / code`, `tax_rate.code`, `payment_term.name`, `payment_method.name`, `txn_class(parent_id,name)` NULLS NOT DISTINCT, `price_list.name` + default-partial, `contact.display_name / tax_id` (partial). - Хужжат рақамлари (14 та): `journal_entry.entry_number`, `invoice_number`, `receipt_number`, `bill_number`, `(vendor_id, vendor_invoice_number)` partial, `payment_number`, `estimate_number`, `po_number`, `cm_number`, `vc_number`, `rr_number`, `sr_number`, `txn_number`, `allocation_number`, `adj_number`, `wtr_number`, `run_number`, `payp_number`. - Механизм: `document_sequence.document_type` → `(tenant_id, document_type)`; `journal_entry(source_module,`

`source_document_id`) partial; `stock_balance(warehouse_id,item_id)` ;
`bank_reconciliation(account_id,statement_date)` ; `payroll_run(period)` posted-partial; allocation'лар
`(payment_id, invoice_id)` каби жуфтликлар - булар FK орқали аллақачон tenant'га боғланган, лекин бир
хиллик учун барибир tenant_id билан кенгайтириш T1 картасида ҳал қилинади (FK-транзитив бўлганлари
теккизилмаслиги мумкин - карта аниқлайди). - `company_settings.singleton_guard` БЕКОР →
`UNIQUE(tenant_id)` . - `app_user.username` ГЛОБАЛ ҚОЛАДИ (login глобал). - `exchange_rate` да unique ЙЎҚ
(append-only, 033) - фақат tenant_id устун + индекс.

3.4 Энг катта риск: JdbcClient хом SQL (10+ service)

Ҳисоботлар Hibernate'ни атайлаб четлаб ўтади: LedgerDashboardService (~9 сўров), TrialBalance, BalanceSheet, ProfitAndLoss (+ByClass), Statement, AccountTransactions, InventoryValuation, GlobalSearch, AttachmentService (EXISTS), ContactService, PayrollRegisterService, FactoryResetService. @TenantId native SQL'ни ФИЛЬТРЛАМАЙДИ - шунинг учун RLS мажбурий (қатлам 2), устига ҳар иссиқ сўровга explicit `tenant_id = :tenantId` предикати ҳам қўшилади (индекс/planner учун).

3.5 Бошқа кросс-кесим нуқталар

- **Scheduler биттагина:** `ExchangeRateScheduler.importDaily` - tenant бўйлаб айланадиган бўлади (T4).
- **Seed икки жойда:** Liquibase (валюта UZS/USD, MAIN омбор, QQS12/ NO_TAX, бирликлар, payment term/method, document_sequence, payroll счётлари) + ApplicationRunner'лар (AdminUserInitializer, DefaultChartInitializer - 51 счёт CSV, DefaultUnitsInitializer). Янги tenant учун буларнинг ҲАММАСИ `TenantProvisioningService` га кўчади (T4); Liquibase seed'лар мавжуд default tenant'га backfill бўлади, янги глобал seed ёзилмайди.
- **Attachment локал диск, ягона каталог** → `tenants/<tenantId>/ой/UUID.ext` + download'да prefix текшируви (T4).
- **ҚИЗИЛ БАЙРОҚ - FactoryResetService:** TRUNCATE/DELETE билан бутун базани тозалайди. Multi-tenant'да TRUNCATE = БАРЧА компаниялар маълумотини ўчириш! RLS ҳам TRUNCATE'ни қатор бўйича филтрламайди. T4'да тўлиқ per-tenant DELETE'га қайта ёзилади; унеча (T1'дан бошлаб) бу экран фақат тизимда битта tenant бўлсагина ишлашига қоровул қўйилади (BR-TEN-*).
- **ЈТЕ узатиш нуқтаси тайёр:** `GlobalModelAttributes` (@ControllerAdvice) - currentTenant/availableTenants/tenantContextVersion шу ердан марказий узатилади; ҳар мутация формада турган `@template.shared.csrf` partial'ига `_tenantContextVersion` hidden input қўшилади - БИТТА partial ўзгаради, ҳамма форма қамралади (plan-а'даги «ҳар формага қўл билан» иши бизда бир нуқтага йиғилади).
- **Тестлар:** ~110 @SpringBootTest, локал `averpo_test` , TestDbSafetyGuard (URL `_test` мажбурий), drop-first. Тест инфраструктурасига «default test tenant + TenantContext» қуриладиган умумий support қўшилади (T1) - тестлар оммавий таҳрир қилинмайди.

4. Асосий қарорлар

- **D1. Column-based, битта база/схема.** Schema-per-tenant РАД (Liquibase/deploy мураккаблиги, QBO ҳам shared).
- **D2. Session resolution** (plan-а тўлиқ). URL'лар ўзгармайди. Frontend'дан келган tenant ID ҳеч қачон ишончли манба эмас.
- **D3. Currency ва ExchangeRate per-tenant.** QBO паритети: ҳар компанияда ўз валюта рўйхати, active флаглари, курслари. ЦБ импорти ҳар tenant'га ўз ёзувини ёзади (қатор кичик, ҳажм арзон). Бу

PostingService'нинг «Money ISO коди каталога қарши текширилади» қондасини per-tenant сақлайди.
(Альтернатива - глобал каталог + per-tenant активация - РАД: икки хил эгалик модели, кўпроқ шохлаш.)

- **D4. Membership модели** (§2). `app_user.role` устуни миграция билан `tenant_membership.role` га кўчади ва ўчирилади.
- **D5. Authorities фаол tenant'дан** (§2) - switch'да Authentication қайта қурилади. `RolePermissions` матрицаси ўЗГАРМАЙДИ.
- **D6. TenantContext (ThreadLocal) shared да туради** - `Uuid7` прецеденти: ledger ҳам ишлатади, модул боғлиқлик йўналиши бузилмайди. `get` fail-fast (tenant йўқ бўлса exception - жим null қайтариш маълумот оқишининг манбаи). `clear` filter'нинг finally'сида. Tenant/TenantMembership entity + web - янги `tenant` модули (`com.averpo.erp.tenant`).
- **D7. Hibernate native @TenantId** : янги `TenantAwareEntity extends BaseEntity` (@MappedSuperclass, tenant_id updatable=false); 64 entity механик равишда унга ўтади. Глобаллар (§3.2) BaseEntity'да қолади. `CurrentTenantIdentifierResolver` TenantContext'дан ўқийди. Аниқ property номлари BOM'даги ҳақиқий Hibernate версиясига қарши текширилади (тахмин қилинмайди).
- **D8. RLS FORCE + DB роль ажратиш**: application энди ЭГАСИ БЎЛМАГАН `averpo_app` роли билан уланади (эгалик ва Liquibase - `averpo` да қолади; owner RLS'ни четлаб ўтади, FORCE + роль ажратиш иккиси ҳам қилинади). Policy: `USING (tenant_id = current_setting('app.current_tenant')::uuid) WITH CHECK (шу)` . Wiring: DataSource wrapper ҳар checkout'да `SELECT set_config('app.current_tenant',?, false)` (PreparedStatement, конкатенация ТАҚИҚ), tenant йўқ бўлса RESET - глобал жадвалларгагина рухсат қолади. `deploy/server-setup.sh` ва `prod env` (`DB_USERNAME=averpo_app`) янгиланади.
- **D9. Рақамлаш**: мавжуд DocumentSequence қатор-қулф механизми сақланади - per-tenant қаторлар per-tenant serialization'ни бепул беради (plan-с'нинг document_counter жадвали КЕРАК ЭМАС, бизники аллақачон шу шаклда).
- **D10. Audit**: `audit_event.tenant_id` NULLABLE - LOGIN_/LOCKOUT/USER_ каби глобал ҳодисаларда NULL, бизнес ҳодисаларда мажбурий. `/audit-log` фаол tenant ҳодисаларини кўрсатади (§9 O2).
- **D11. Attachment** per-tenant каталог + prefix текшируви (§3.5).
- **D12. FactoryReset** per-tenant DELETE'га қайта ёзилади; TRUNCATE multi-tenant базада умуман ТАҚИҚ (§3.5 қизил байроқ).
- **D13. MDC**: tenant filter MDC'га tenantId ёзади - ҳар log қатори tenant билан (plan-с 5.4).

5. Фазалар

Ҳар фаза = алоҳида карта(лар) тўлқини, охирида `./gradlew test` яшил, тизим deploy қилинадиган ҳолатда. Changeset рақамлари 069'дан.

T0 - Спец ва каталог (код йўқ)

- `docs/modules/multi-tenancy.md` спец (шу план асосида, тасдиқдан кейин). `engineering-rules.md`'га янги темир қоида номзоди: «янги бизнес entity TenantAwareEntity'дан мерос олади; native SQL tenant_id предикатсиз ёзилмайди ёки tenant-free деб изоҳланади».
- `docs/business-rules.md` : BR-TEN-001.. (tenant ACTIVE эмас, membership йўқ, контекст версияси эскирган, cross-tenant мурожаат, factory reset кўп-tenant'да ва ҳ.к.).

T1 - Схема + Hibernate (тизим ҳали бир-компаниялик бўлиб ишлайди)

- Changeset'лар (069+): `tenant` жадвали; барча 64 жадвалга `tenant_id UUID` NULLABLE; default tenant INSERT (номи `company_settings.name`'дан); backfill UPDATE (64 жадвал); NOT NULL + FK; §3.3 unique'ларини қайта қуриш; иссиқ индексларни `tenant_id` БИРИНЧИ устун билан қайта қуриш (тўлиқ индекс инвентаризацияси карта ичида; энг муҳими `journal_entry_line`). Prod'да катта жадвал йўқ - batched backfill шарт эмас, лекин UPDATE'лар алоҳида changeset'да.
- `TenantAwareEntity` + 64 entity кўчиши (механик, лекин катта diff - бир нечта картага бўлинади); `SpringTenantIdentifierResolver`.
- Ўткинчи resolver: `TenantContext` бўш бўлса default tenant (константа эмас - `tenant` жадвалидаги ягона қатор) қайтарилади. Шу туфайли UI/login/тестлар ҳали ЎЗГАРМАЙДИ, ҳаммаси default tenant ичида ишлайверади. Бу «фиксация» T2'да олиб ташланади.
- Тест support: default test tenant + контекст ўрнатувчи умумий асос.
- FactoryReset'га «фақат битта tenant бўлса» қоровули (BR-TEN-*).

T2 - Контекст, membership, UI

- `tenant` модули: `Tenant/TenantMembership` entity + service + web.
- Changeset: `tenant_membership` + миграция (`app_user.role` → default tenant membership'и) + `app_user.role` DROP.
- `TenantResolutionFilter` (OncePerRequestFilter, authentication'дан КЕЙИН): session'дан tenant ID → мавжудлик → ACTIVE → membership → `TenantContext.set` → finally clear. plan-а'даги 8 текширув тўлиқ. T1'даги default-fallback ОЛИБ ТАШЛАНАДИ - tenant-required йўлда контекстсиз сўров fail-fast.
- Endpoint таснифи марказий: tenant-free рўйхат (`/login` , `/logout` , `/tenant/**` , статик, `/error` , `/telegram/webhook`) + қолгани tenant-required; тест билан қотирилади.
- Login оқими (§2), компания танлаш/яратиш саҳифалари, `POST /tenant/switch` (server-side membership текшируви, версия++, НТМХ'да НХ-Redirect, оддийда 303).
- `GlobalModelAttributes` : `currentTenant`, `availableTenants`, `tenantContextVersion`. Sidebar'га компания танлагич (мобилда drawer ичида, 375px қондаси).
- `shared/csrf` partial'ига `_tenantContextVersion` hidden input; мутацияларда версия текшируви filter'да - мос келмаса 409 + «Компания ўзгарган. Саҳифани янгиланг» (BR-TEN-*).
- Authorities switch'да қайта қурилади (D5).

T3 - RLS (иккинчи қатлам)

- Changeset: 64 жадвалга ENABLE + FORCE ROW LEVEL SECURITY + policy.
- `TenantAwareDataSource` (set_config/RESET wiring, D8).
- `averpo_app` роли: server-setup.sh, GRANT'лар, prod env; локал dev/ test учун ҳам айнан шу роль модели (тест RLS'ни РЕАЛ текшириши учун)
- `averpo_test`'да ҳам app роли эгалик қилмайди).
- Изоляция тестлари (§6, RLS қисми). TestDbSafetyGuard сақланади.

T4 - Кросс-кесим хизматлар

- `TenantProvisioningService` : янги компания яратилганда - Tenant + CompanySettings + яратувчига SUPER_ADMIN membership + COA (51 счёт CSV) + бирликлар + валюта (UZS/USD) + MAIN омбор +

QQS12/NO_TAX + payment term/method + document_sequence қаторлари. Бир транзакцияда, ҳаммаси ўша tenant контекстида.

- **ExchangeRateScheduler** : ACTIVE tenant'лар бўйлаб айланиш (**TenantIterator** helper: set → иш → finally clear); аудит ҳар tenant'нинг ўзига.
- Attachment: per-tenant каталог + мавжуд файлларни default tenant каталогига кўчириш скрипти + download prefix текшируви.
- Audit: tenant_id ёзиш (AuditLogService), /audit-log фильтри.
- FactoryReset: per-tenant DELETE (FK тартибида), TRUNCATE бекор.
- GlobalSearch/Excel import/AttachmentService EXISTS сўровлари: explicit tenant предикат ревизияси.
- MDC wiring (D13).

T5 - Қаттиқлаштириш, ҳужжат, релиз

- §6 тест матрицаси тўлиқ яшил. ArchUnit услубидаги қоровул: tenant-scoped доменда **TenantAwareEntity** дан мерос олмаган entity build'ни йиқитади; nativeQuery/JdbcClient ишлатган янги метод «tenant-safe» изоҳисиз ўтмайди (реализация шакли картада).
- engineering-rules.md темир қоида, architecture.md, тегишли модул спецлари янгиланади. Runbook: янги компания очиш, suspend қилиш, support учун cross-tenant киришнинг хавфсиз йўли.
- Prod миграция (§7).

6. Тест матрицаси (мажбурий, plan-с'дан Averpo'га мослаб)

Ҳаммаси реал PostgreSQL **averpo_test** да (RLS H2'да йўқ - бизда H2 ҳам йўқ):

1. **Hibernate изоляцияси**: A tenant'да ёзилган маълумот В контекстда ҳеч қайси repository сўровида кўринмайди; В контекстда А'нинг **findById** 'си бўш (Hibernate версиясининг find/tenant хатти-ҳаракати тахмин қилинмай, айнан тест билан исботланади).
2. **Фақат RLS**: хом JDBC билан (Hibernate четлаб) **app.current_tenant=B** ҳолатда А қаторларига SELECT/UPDATE/DELETE - 0 қатор.
3. **FORCE текшируви**: owner роли билан ҳам RLS амал қилади.
4. **WITH CHECK**: session В бўлганда tenant_id=A INSERT рад этилади.
5. **Контекст оқиши**: thread-pool қайта ишлатилганда кейинги контекстсиз сўров fail-fast (А маълумотини жим кўрмайди).
6. **Unique кўлами**: икки tenant'да бир хил invoice_number - иккиси ҳам ўтади; битта tenant ичида дубль - constraint хатоси.
7. **Рақамлаш**: битта tenant'да параллел post - кетма-кет рақам, deadlock йўқ; икки tenant параллел - бир-бирини блокамайди.
8. **Хом SQL ҳисоботлар**: Trial Balance/BS/P&L иккита tenant'да фақат ўз рақамларини кўрсатади (JdbcClient йўли RLS орқали).
9. **Scheduler**: ЦБ импорти ҳар ACTIVE tenant'га ўз ёзуви/аудити.
10. **Stale form**: эски **_tenantContextVersion** билан POST - 409, ҳеч нарса сақланмайди.
11. **Ledger инварианти** (темир қоида 4/7): ҳар tenant ичида $\text{sum}(\text{debitBase}) = \text{sum}(\text{creditBase})$ - мавжуд posting тестлари tenant контекстда ҳам яшил.

7. Prod миграция (deploy режаси - фақат фойдаланувчи буйруғи билан)

1. Тўлиқ backup (`pg_dump | gzip` , DEPLOY-LOG қондаси).
2. `averpo_app` ролини яратиш + GRANT (server-setup.sh янгиланган).
3. Deploy: Liquibase 069+ (tenant, backfill - проддаги ягона компания default tenant бўлади, ҳамма қатор унга бирикади), env'да `DB_USERNAME=averpo_app` .
4. Smoke: login → мавжуд компания автотанланади → dashboard/ҳисобот рақамлари деплойдан олдингиси билан АЙНАН тенг (Trial Balance солиштируви) → аудитда ҳодисалар tenant билан.
5. Rollback: jar'ни олдинги релизга қайтариш + backup restore (changeset'лар rollback'сиз ёзилади - қайтиш фақат restore).

8. Кўламдан ТАШҚАРИДА (ҳозир қилинмайди)

- Schema-per-tenant / database-per-tenant.
- Partitioning (келажакда journal_entry_line ой бўйича - RLS/unique дизайни бунга зид қилинмайди: partition калити unique'ларга киришини T1 картаси ҳисобга олади).
- Billing/тарифлар (`plan` устуни ҳам ЙЎҚ - зарур бўлганда алоҳида).
- Ўзини-ўзи рўйхатдан ўтказиш (self-signup) UI ва email invite.
- Subdomain routing.
- Platform back-office (Averpo staff учун барча tenant'ларни бошқариш экрани) - \$9 О4.

9. Очиқ саволлар (фойдаланувчи қарори керак)

- **01. Фойдаланувчи яратиш модели:** username глобал unique. Компания admin'и янги фойдаланувчи очганда у ГЛОБАЛ аккаунт очади (бошқа компания ҳам кейин шу одамни аъзо қила олади)ми, ёки мавжуд username'ни аъзо қилиб қўшиш (invite'сиз, тўғридан-тўғри) оқимими? Тавсия: v1'да иккиси ҳам SUPER_ADMIN экранида - «янги аккаунт очиш» ва «мавжуд username'ни аъзо қилиш»; email invite йўқ.
- **02. Auth аудити кўриниши:** LOGIN_*/LOCKOUT tenant'сиз ҳодисалар - улар қайси /audit-log'да кўринади? Тавсия: фаол tenant аъзоларининг auth ҳодисалари шу tenant журналида ҳам кўринади (username бўйича join), tenant_id'сиз сақланади.
- **03. Telegram plugin:** ҳозир bot token глобал singleton. v1'да глобал қолдирамизми (хабарларда компания номи билан) ёки ҳар компанияга ўз боти? Тавсия: глобал қолсин, per-tenant bot - кейин.
- **04. Platform back-office қачон:** tenant suspend/кўриш учун дастлаб SQL/runbook кифоями? Тавсия: ҳа, v1 runbook; экран - кейин.
- **05. D3 (Currency per-tenant) тасдиқи** - QBO паритети бўйича тавсиямиз шу, лекин глобал каталог альтернативаси ҳам мумкин.

10. Тахминий кўлам

- Changeset'лар: ~8-12 та (069..080 оралиғи).
- Entity таҳрири: 64 та (механик, TenantAwareEntity + import).
- Янги Java: tenant модули (~6-8 класс), TenantContext, resolver, filter, datasource wrapper, provisioning, iterator (~12-15 класс).
- Таҳрир: JpaUserDetailsService, SecurityConfig, GlobalModelAttributes, csrf partial, sidebar, ExchangeRateScheduler, AttachmentService, FactoryResetService, AuditLogService, 10+ report service (explicit

предикат), UserService/users экрани (membership).

- Тестлар: янги изоляция тўплами (§6) + мавжуд ~110 класс тест support орқали яшил сақланади.
- Тартиб: T1 ва T2 энг катта тўлқинлар, ҳар бири бир нечта карта; жараён мавжуд review оқимида (coder'лар + reviewer'лар + арбитр).